

电子设计基础训练 ——音频输出实验

北京航空航天大学
电子信息工程学院





音频输出实验

- 数字音频转换器PCM5102
- 集成电路内置音频总线I2S
- 实验1：基本波形输出
- 实验2：左右声道波形输出
- 实验3：扬声器播放单音信号
- 实验4：音频数据播放
- 实验5：双声道放音系统



数字音频转换器PCM5102

■ 能把数字信号转换为模拟信号的电路称为数模转换器(DAC)

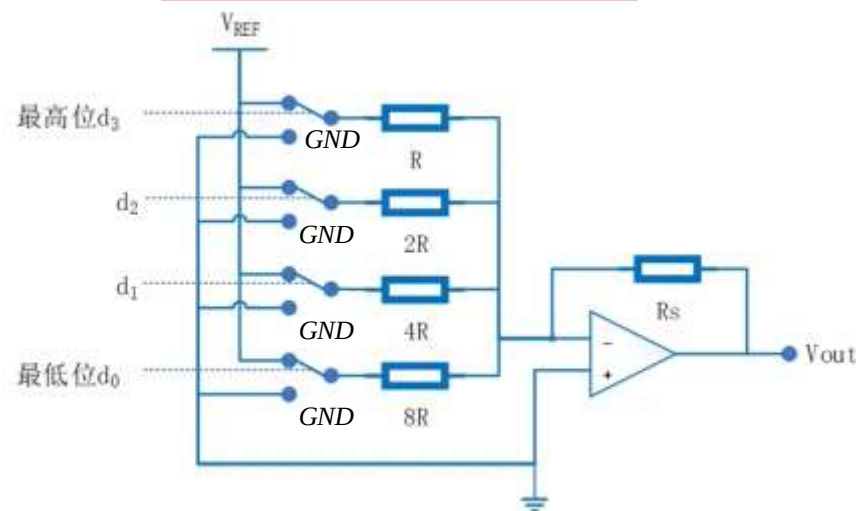
➤ 4位二进制输入，对应16个量化电平，
不同位二进制数对电压的贡献不同

➤ 如0001 ↔ $V_{ref}/16$, 0100 ↔ $4 * V_{ref}/16$

➤ 利用开关、电阻和运算放大器，即可实现数字信号0和1（开关的打开和闭合），到模拟电压输出的转换



d_3	d_2	d_1	d_0
$\frac{8}{16}V_{ref}$	$\frac{4}{16}V_{ref}$	$\frac{2}{16}V_{ref}$	$\frac{1}{16}V_{ref}$





数字音频转换器PCM5102

■ 高性能数字音频转换器

➤ I2S接口信号线:

位时钟BCK

声道选择LRCK

数据DIN

注: I2S模式下, SCK 系统时钟信号线不用, 在实验中SCK 接地。

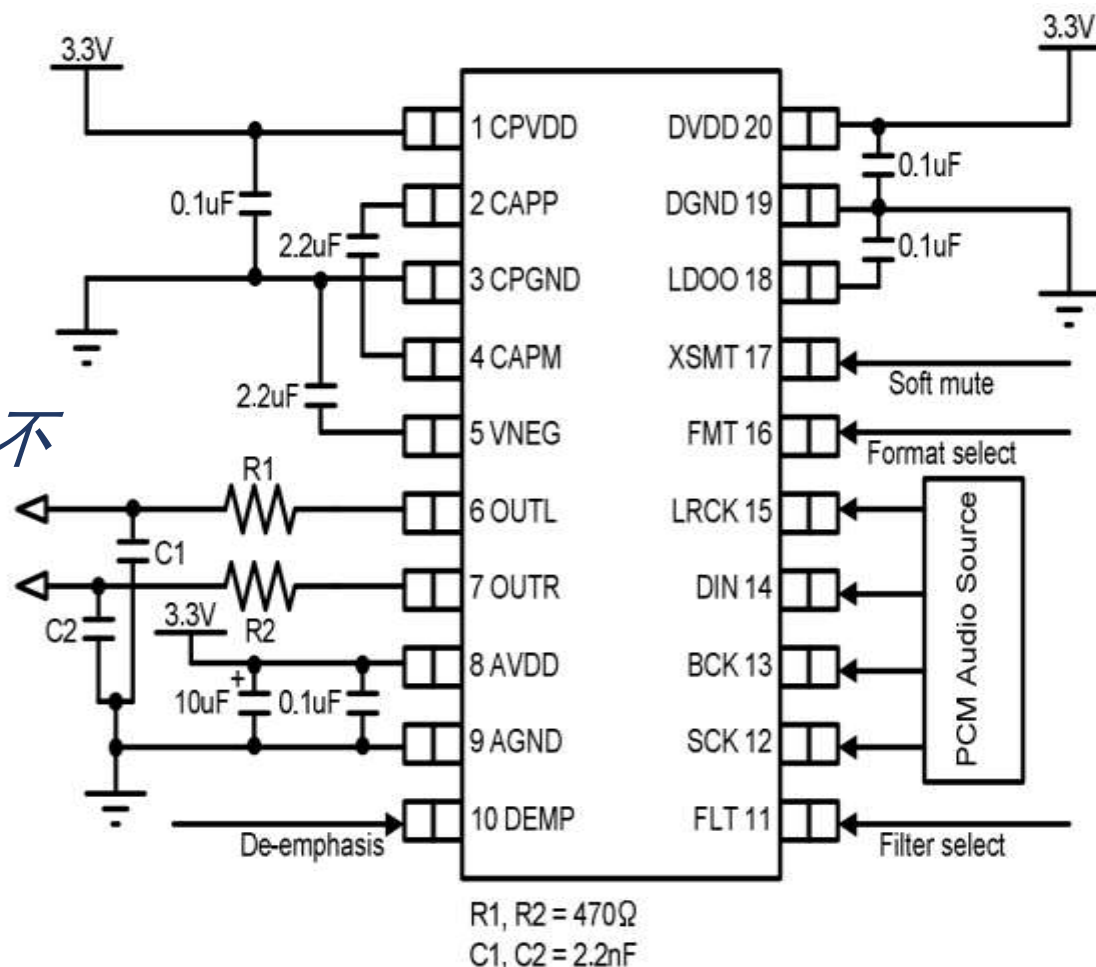
➤ 配置引脚:

FLT: 滤波器选择

DEMP: 采样率去加重控制

XSMT: 静音控制 (接高)

FMT: 格式选择





集成电路内置音频总线I2S

■ I2S (Inter-IC Sound) 是一种应用于数字音频传输的串行接口标准

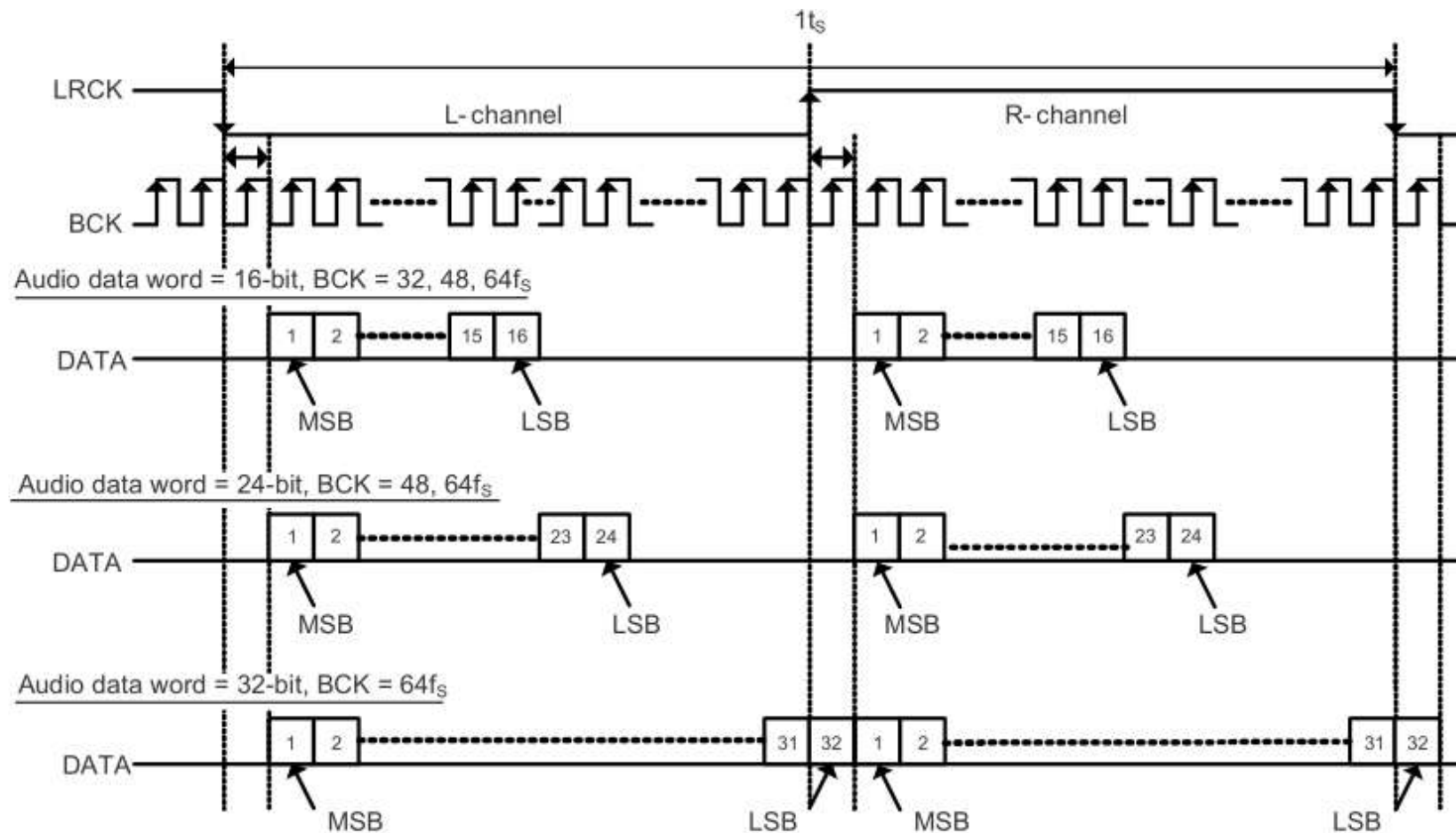
➤ I2S协议只定义三根信号线：

- 时钟线 (Continues Serial Clock, SCK) : SCK线提供了同步音频数据传输的时钟信号。确定了数据传输的速度和时序。该时钟也称为 **Bit Clock (BCK)** 。 $BCK \text{ 的频率} = 2 \times \text{采样频率} \times \text{位宽}$
- 左/右声道线 (Left-Right Clock, LRCK) : LRCK线指示了当前传输的是左声道的音频数据还是右声道的音频数据。它也被称为字选择信号 **Word Select (WS)** 。 $LRCK \text{ 的频率} = \text{采样频率}$
- 数据线 (Serial Data, SD) : SD线用于传输实际的音频数据。数据的位宽可以根据具体应用而变化,通常为16位或32位。TX方向为: Serial Data Out (SDOUT) ; RX方向为: Serial Data In (SDIN) 。



集成电路内置音频总线I2S

■ 通信格式



I²S Data Format; L-channel = LOW, R-channel = HIGH



音频输出实验

- 实验1：基本波形输出
- 实验2：左右声道波形输出
- 实验3：扬声器播放单音信号
- 实验4：音频数据播放
- 实验5：双声道放音系统



实验1：基本波形输出

■ 认识实验板

➤ 音频输出板

- 供电VCC (5V) ,GND
- I2S数据线
 - LRCK,DIN,BCK
- 配置
 - FLT,DEMP,XSMT,FMT
- 音频输出
 - OUTL,OUTR,耳机孔
- 系统时钟
 - SCK接GND

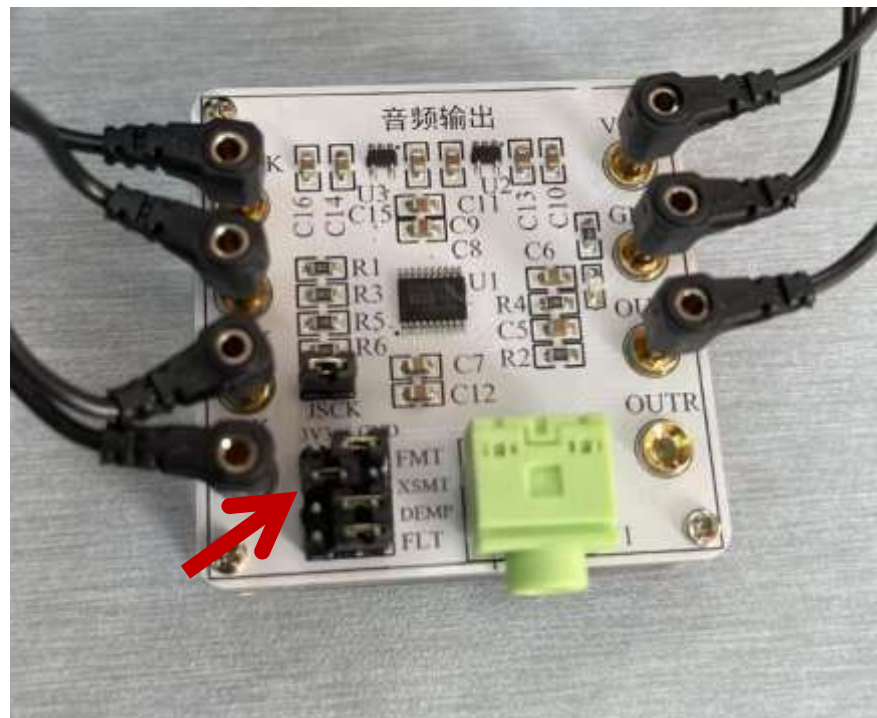




实验1：实物连线

■ 连接ESP32-C3和音频输出板

- +5V—VCC
- GND—GND
- 1—DIN
- 2—BCK
- 3—LRCK
- GND—SCK
- 音频输出板上配置引脚XSMT通过跳线帽连接到3V3上
- 示波器观察OUTL（左声道）





实验1：实验代码

■ 引入库函数

- i2s库，调用硬件i2s
- math库，调用sin/cos等函数

```
#include <driver/i2s.h>
#include <Arduino.h>
#include <math.h>
```

■ 配置I2S引脚

```
#define I2S_MIC_SERIAL_CLOCK GPIO_NUM_2
#define I2S_MIC_LEFT_RIGHT_CLOCK GPIO_NUM_3
#define I2S_MIC_SERIAL_DATA GPIO_NUM_1

i2s_pin_config_t i2s_mic_pins = {
    .bck_io_num = I2S_MIC_SERIAL_CLOCK, // 设置I2S时钟引脚（串行时钟）
    .ws_io_num = I2S_MIC_LEFT_RIGHT_CLOCK, // 设置I2S左右声道时钟引脚
    .data_out_num = I2S_MIC_SERIAL_DATA, // 设置I2S数据输出引脚
    .data_in_num = I2S_PIN_NO_CHANGE // 不改变数据输入引脚（此处未使用数据输入）
};
```



实验1：实验代码

■ I2S接口参数

```
i2s_config_t i2s_config = {  
    .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_TX), // 设置I2S为主模式（MASTER），并指定为发送（TX）模式  
    .sample_rate = SAMPLE_RATE, // 设置采样率（8000）  
    .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT, // 每个样本使用16位  
    .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT, // 设置为单声道输出，只输出左声道  
    .communication_format = I2S_COMM_FORMAT_I2S, // 设置为I2S通信格式  
    .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1, // 中断优先级为LEVEL1  
    .dma_buf_count = 4, // DMA缓冲区的数量为4  
    .dma_buf_len = 1024, // 每个DMA缓冲区的长度为1024  
    .use_apll = false, // 不使用应用级PLL（直接使用默认时钟）  
    .tx_desc_auto_clear = false, // 关闭自动清除发送描述符  
    .fixed_mclk = 0 // 不使用固定的主时钟频率  
};
```

■ setup中配置I2S

```
// 配置I2S  
i2s_driver_install(I2S_NUM_0, &i2s_config, 0, NULL);  
i2s_set_pin(I2S_NUM_0, &i2s_mic_pins);
```



实验1：实验代码

■ 生成波形查找表

```
if(mode==0){// 生成正弦波
    for (uint32_t i = 0; i < LENGTH; i++) {
        // 计算正弦波值
        float sample = AMPLITUDE * sin(2 * PI * FREQUENCY * i / SAMPLE_RATE);
        i2sBuf[i] = (int16_t)(sample);
    }
}
else if(mode==1){// 生成锯齿波
    for (uint32_t i = 0; i < LENGTH; i++) {
        // 计算锯齿波值
        float sample = (float)AMPLITUDE * i / LENGTH;
        i2sBuf[i] = (int16_t)(sample-AMPLITUDE/2);
    }
}
else if(mode ==2){// 生成方波
    for (uint32_t i = 0; i < LENGTH/2; i++) {
        i2sBuf[i] = -1*AMPLITUDE;
        i2sBuf[i+LENGTH/2] = AMPLITUDE;
    }
}
```



实验1：实验代码

■ i2s_write()函数

```
// 写入数据到I2S
```

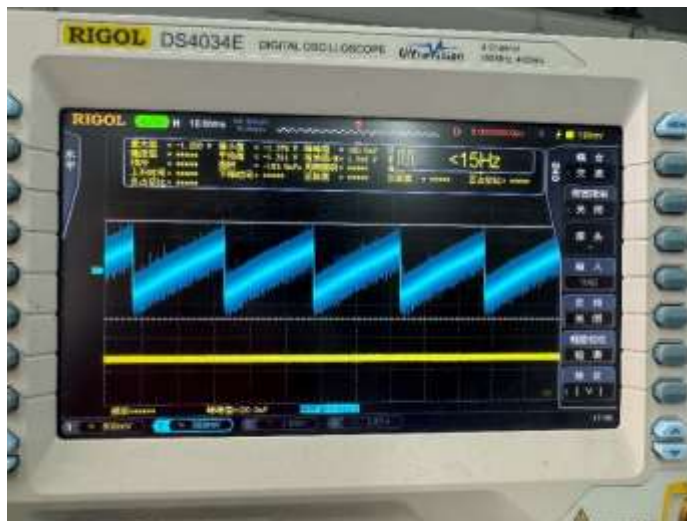
```
i2s_write(I2S_NUM_0, i2sBuf, bytes, &i2sWriteLen, portMAX_DELAY);
```

- 参数1: i2s设备号, ESP32C3中只有一个i2s设备, 编号为0
- 参数2: 数据源指针, 使用i2sBuf数组作为数据源
- 参数3: 一次传输数据的字节数 (数据类型size_t)
- 参数4: 返回实际写入 I2S 硬件的字节数 (数据类型size_t)
- 参数5: 超时等待



实验1：实验任务

- 用示波器观察音频输出板上OUTL引脚输出信号，修改波形类型mode和频率FREQUENCY，观察波形和频率





音频输出实验

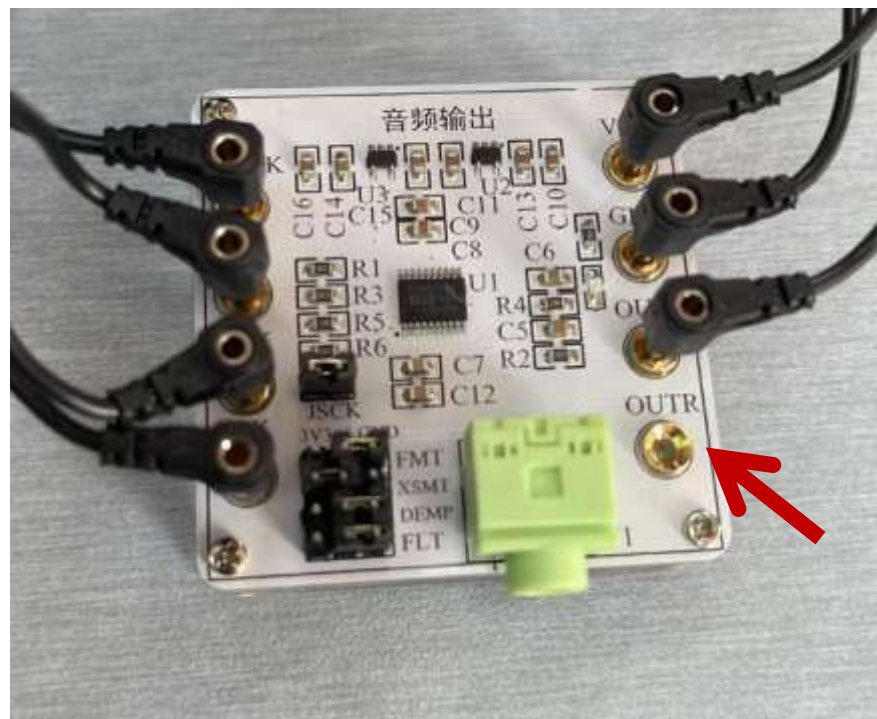
- 实验1：基本波形输出
- 实验2：左右声道波形输出
- 实验3：扬声器播放单音信号
- 实验4：音频数据播放
- 实验5：双声道放音系统



实验2：实物连线

■ 连接ESP32-C3和音频输出板

- +5V—VCC
- GND—GND
- 1—DIN
- 2—BCK
- 3—LRCK
- GND—SCK
- 音频输出板上配置引脚XSMT通过跳线帽连接到3V3上
- 示波器同时观察OUTL（左声道）和OUTR（右声道）





实验2：实验代码

■ 配置I2S

```
i2s_config_t i2s_config = {  
    .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_TX),  
    .sample_rate = SAMPLE_RATE,  
    .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,  
    .channel_format = I2S_CHANNEL_FMT_RIGHT_LEFT, // 配置为双声道  
    .communication_format = I2S_COMM_FORMAT_I2S,  
    .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,  
    .dma_buf_count = 4,  
    .dma_buf_len = 1024,  
    .use_apll = false,  
    .tx_desc_auto_clear = false,  
    .fixed_mclk = 0};
```



实验2：实验代码

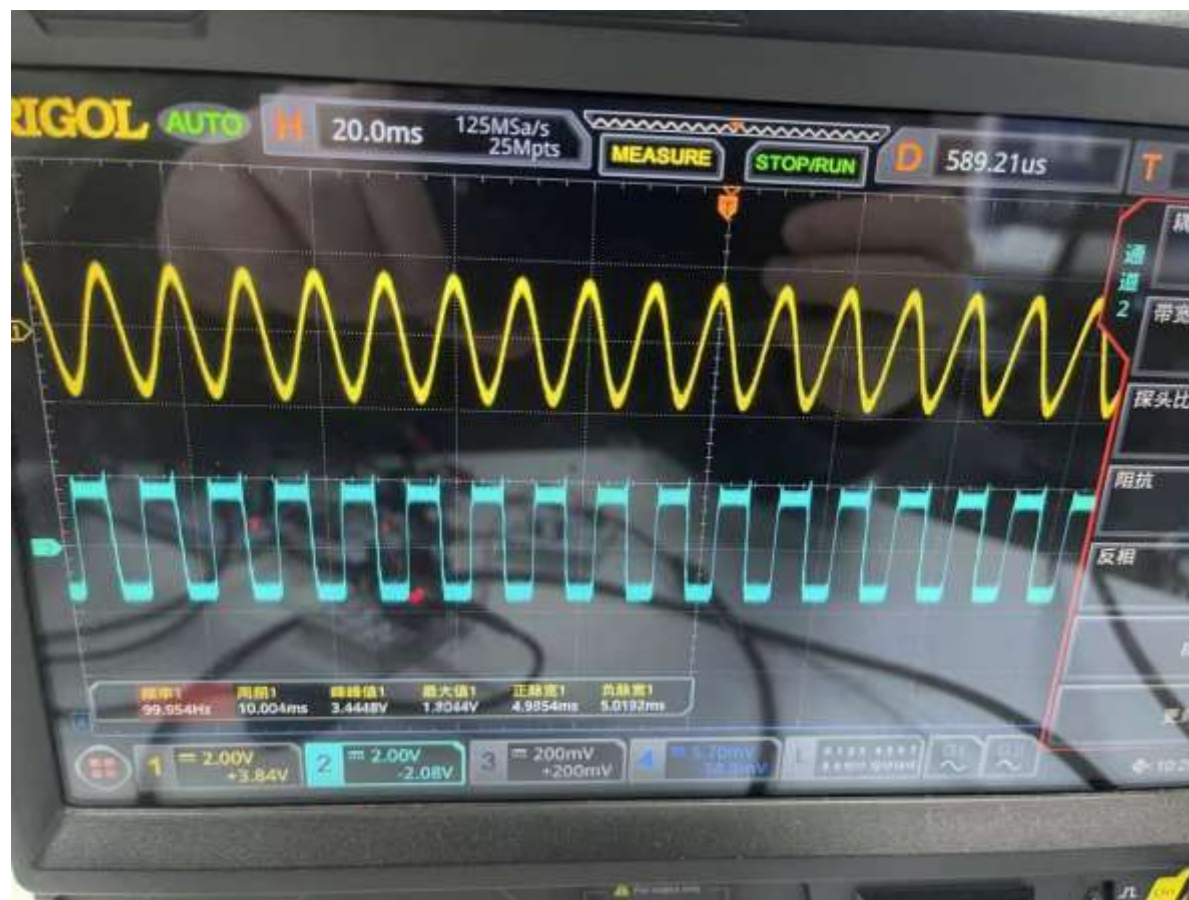
■生成双声道波形查找表

```
for (uint32_t i = 0; i < LENGTH; i++) {  
    // 左声道: 正弦波  
    float sine_sample = AMPLITUDE * sin(2 * PI * FREQUENCY * i / SAMPLE_RATE);  
    i2sBuf[i * 2] = (int16_t)(sine_sample); // 左声道数据  
  
    // 右声道: 方波  
    if (i < LENGTH / 2) {  
        i2sBuf[i * 2 + 1] = -1*AMPLITUDE; // 方波低电平  
    } else {  
        i2sBuf[i * 2 + 1] = AMPLITUDE; // 方波高电平  
    }  
}  
  
// 写入数据到I2S  
i2s_write(I2S_NUM_0, i2sBuf, bytes * 2, &i2sWriteLen, portMAX_DELAY);
```



实验2：实验任务

- 使用双踪示波器分别观察OUTL和OUTR波形，应观察到不同波形





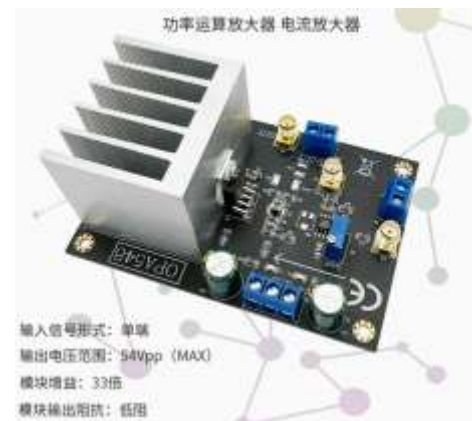
音频输出实验

- 实验1：基本波形输出
- 实验2：左右声道波形输出
- **实验3：扬声器播放单音信号**
- 实验4：音频数据播放
- 实验5：双声道放音系统



音频功放

- 功率放大器简称“功放”，是指在给定失真率条件下能产生最大的功率输出的放大器
- 用于驱动阻性负载（扬声器）、容性负载（压电陶瓷）、感性负载（电磁线圈）
- 可分为A类、B类、AB类、D类
 - A类：无失真，效率20%-30%
 - B类：有交越失真，效率可达78.5%
 - AB类：结合AB类优点，无失真，效率高
 - D类：数字功率放大器，效率可达80%以上
低失真、工作复杂





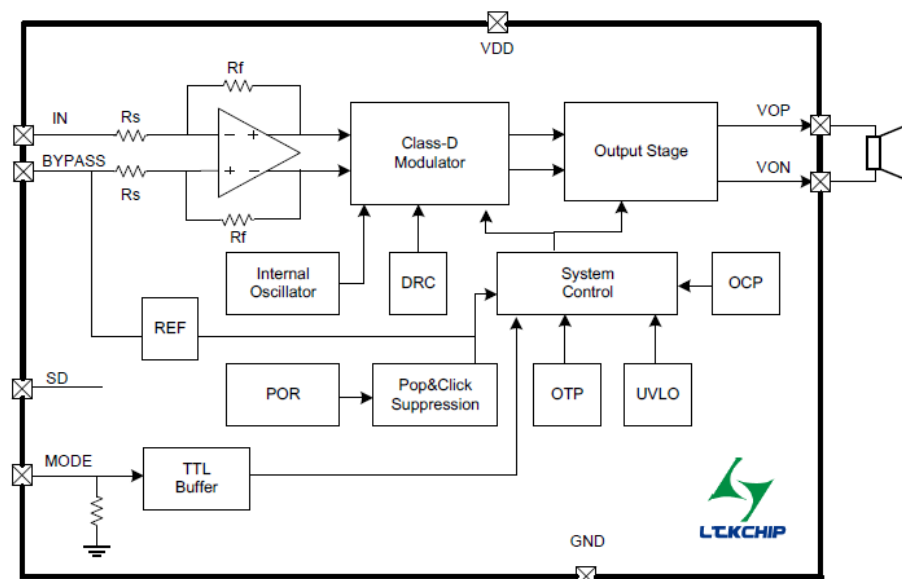
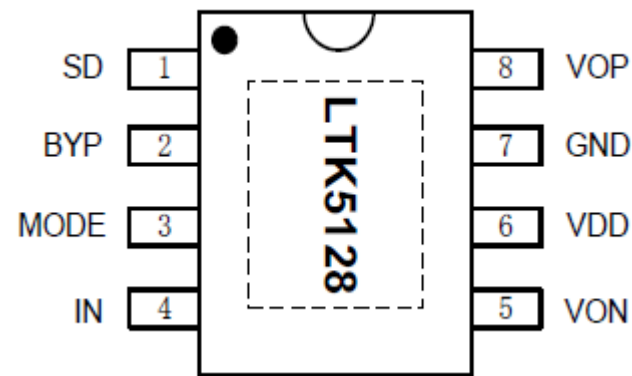
音频功放

■ LTK5128

➤ 一款高性能、多功能、低功耗的AB类/D类工作模式切换音频功率放大芯片

➤ 8引脚

- SD，关断控制，高电平有效。当输入高电平时，芯片将被关断
- BYP，内部共模参考电压该。管脚提供内部共模参考电压
- MODE，高电平D类，低电平AB类，默认是AB类该管脚控制芯片的工作模式管。
- IN，模拟输入端，反相，该管脚接收输入信号
- VDD，为芯片提供电源，最大6V
- GND地，电源地
- VOP模拟输出正极
- VON模拟输出复极





音频功放

■ LTK5128

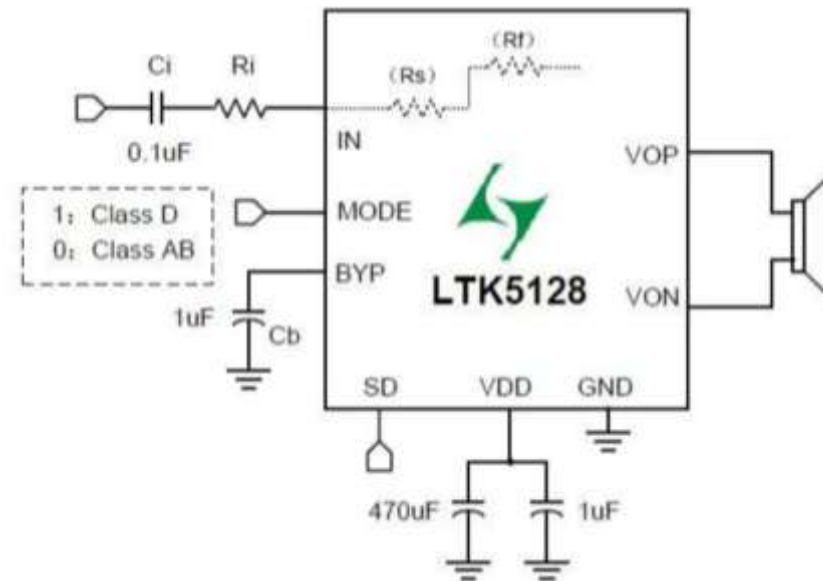
- 放大增益增益由内部电阻 R_f 和 R_s 以及外接电阻 R_i 决定

$$A_v = 20 \log\left(\frac{R_f}{R_i + R_s}\right)$$

$$R_s = 6.5k\Omega$$

$$R_f = 195k\Omega$$

- SHD: 默认接GND, 若接到VCC, 静音
- MODE: 默认接GND (AB类)
- IN: 音频输入
- ING: 输入GND, 与GND自动连接
- J1: 扬声器接口, P接红线、N接黑线





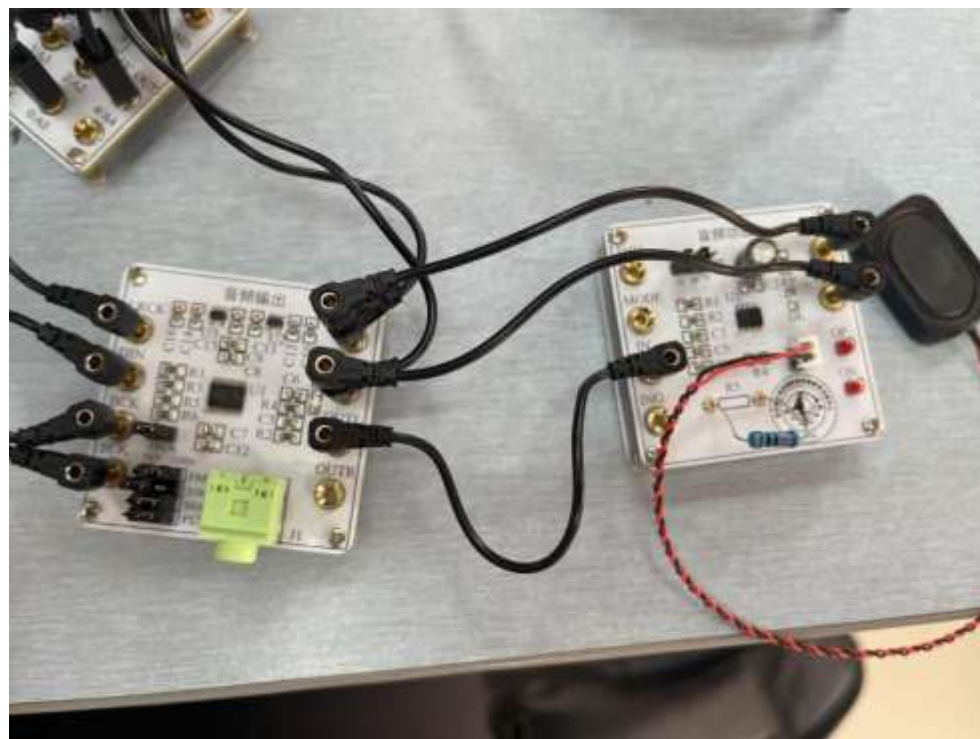
实验3：实物连线

■ 连接ESP32-C3和音频输出板

- +5V—VCC
- GND—GND
- 1—DIN
- 2—BCK
- 3—LRCK
- GND—SCK

■ 连接音频功放板和音频输出板

- +5V—VCC
- GND—GND
- IN—OUTL





实验3：实验原理

- 利用扬声器播放单音信号，通过修改频率改变音高。以下是各标准音高对应的频率。

国际标准音高与频率对照表 (Scientific Pitch Notation)												
频 率 度 音 符	C(do)	C [#] /D ^b	D(re)	D [#] / ^b E	E(mi)	F(fa)	F [#] /G ^b	G(so)	G [#] /A ^b	A(la)	A [#] / ^b B	B(si)
0	16.352	17.324	18.354	19.446	20.602	21.827	23.125	24.500	25.957	27.501	29.136	30.868
1	32.704	34.649	36.709	38.892	41.204	43.655	46.250	49.001	51.914	55.001	58.272	61.737
2	65.408	69.297	73.418	77.784	82.409	87.309	92.501	98.001	103.829	110.003	116.544	123.474
3	130.816	138.595	146.836	155.567	164.818	174.618	185.002	196.002	207.657	220.005	233.087	246.947
4	261.632	277.189	293.672	311.135	329.636	349.237	370.003	392.005	415.315	440.010	466.175	493.895
5	523.264	554.379	587.344	622.269	659.271	698.473	740.007	784.010	830.629	880.021	932.350	987.790
6	1046.528	1108.758	1174.688	1244.538	1318.542	1396.947	1480.013	1568.019	1661.258	1760.042	1864.699	1975.580
7	2093.056	2217.515	2349.376	2489.076	2637.084	2793.893	2960.027	3136.039	3322.517	3520.084	3729.398	3951.160
8	4186.112	4435.031	4698.751	4978.153	5274.169	5587.787	5920.053	6272.077	6645.034	7040.168	7458.797	7902.319
9	8372.224	8870.062	9397.502	9956.306	10548.337	11175.573	11840.106	12544.155	13290.068	14080.335	14917.594	15804.639



实验3：实验代码

■ 音符与频率转换函数

```
int note_to_index(char note, int octave) {  
    int index = -1;  
    // 计算音符索引：例如 C -> 0, D -> 1, E -> 2, 等等  
    switch (note) {  
        case 'C': index = 0; break;  
        case 'D': index = 2; break;  
        case 'E': index = 4; break;  
        case 'F': index = 5; break;  
        case 'G': index = 7; break;  
        case 'A': index = 9; break;  
        case 'B': index = 11; break;  
        default: index = -1; break; // 如果输入的是无效字符，返回-1  
    }  
    return index + (octave * 12); // 将音符所在的八度 (octave) 计算进去  
}
```




实验3：实验代码

■ 解析音符

```
// 检查串口是否有输入数据
if (Serial.available() > 0) {
    String input = Serial.readString();
    input.trim(); // 去除多余的空格和换行符

    // 解析音符
    if (input.length() >= 2) {
        char note = input.charAt(0); // 音符名 (C, D, E, F, G, A, B)
        int octave = input.charAt(input.length() - 1) - '0'; // 获取音高的 octave
        int noteIndex = note_to_index(note, octave); // 获取对应音符的index

        // 确保音符有效
        if (noteIndex != -1) {
            frequency = note_frequencies[noteIndex];
            Serial.print("Frequency set to: ");
            Serial.println(frequency);
        }
        else {
            Serial.println("Invalid note! Please enter a valid note (e.g., C4, A4).");
        }
    }
}
```

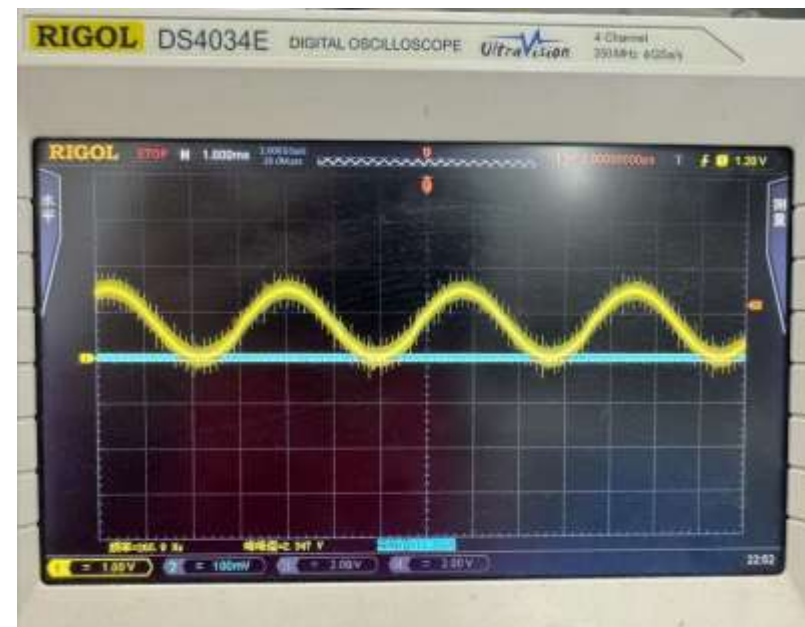


实验3：实验任务

- 在串口监视器输入频率和波形，观察示波器上的波形，并聆听扬声器音频输出

➤ 例如，输入C4，串口监视器返回提示：

```
Frequency set to: 261.63
```



- 通过调音器APP测音符的准确度，对于准确度低的音符，适当修改代码逻辑以更加适配当前选择的输出频率。



实验3：实验任务

- 对代码进行修改，使其不仅可以输出正弦音符，还可以输出方波音符和锯齿音符
 - 例如，输入C4，播放261.63Hz的正弦音符；输入C4S，播放261.63的方波音符；输入C4R，播放261.63的锯齿音符
- 现场检查音符输出的频率准确度和音色
 - 数据长度整数截取后尽可能贴近完整周期更有利于频率稳定性



音频输出实验

- 实验1：基本波形输出
- 实验2：左右声道波形输出
- 实验3：扬声器播放单音信号
- **实验4：音频数据播放**
- **实验5：双声道放音系统**



音频格式

■ 分类

➤ 未压缩音频格式

- 音质佳、体积大
- WAV、AIFF

➤ 无损压缩音频格式

- 不损失音质、体积小
- FLAC、APE

➤ 有损压缩音频格式

- 音质有损、体积很小
- MP3、AAC





音频格式

■ WAV格式

➤ RIFF文件结构

- chunk
- 多个chunk嵌套
- 最外层是一个标识为"RIFF"的类型块chunk1
 - 嵌套了2个chunk
 - chunk2的标识为"fmt"
 - 存储音频属性
 - chunk3的标识为"data"
 - 存储音频PCM数据

```
1. struct chunk
2. {
3.     char    chunkId[4];        // 4 个字符组成的 chunk 标识, 少于 4 字符右补空格
4.     uint32_t chunkSize;        // 数据块的大小, 字节
5.     char    data[chunkSize]
6. };
```

字段				长度(B)	字段描述
chunk1	chunkId			4	第一个chunk的标识始终是"RIFF"
	chunkSize			4	该chunk的数据大小, 包括chunkType
	chunkType			4	对于Wave文件chunk的类型为"WAVE"
	data	chunk2	chunkId	4	Wave文件的第二个chunk标识为"fmt"
			chunkSize	4	该chunk的数据大小
			wFormatTag	2	音频数据格式, 0x0001表示PCM数据
			nChannels	2	声道数
			nSamplesPerSec	4	采样率, 每秒采样次数
			nAvgBytesPerSec	4	每秒的音频数据大小(B), 声道数*采样率*每个采样点的比特数/8
			nBlockAlign	2	每个时刻的音频数据块大小(B), 声道数*每个采样点的比特数/8
			wBitsPerSample	2	每个采样点(的幅值)用多少比特编码(8 or 16)
	data	chunk3	chunkId	4	Wave文件的第三个chunk的标识为"data"
			chunkSize	4	该chunk的数据大小
			data	PCM数据	chunkSize PCM数据



实验4：素材准备

■ WAV文件准备

- 从网络下载或自己录制一段音频，保存为.wav格式或利用格式工厂等软件，将其他音频格式更改为.wav
- 安装软件Audacity，截取一段时间的音频数据，导出音频，格式选择为**单声道**，**signed 16-bit PCM**，采样率可在8kHz-44.1kHz之间选择





实验4：素材准备

■ WAV文件准备

- 安装软件Audacity，截取一段时间的音频数据，导出音频，格式选择为**单声道**，**signed 16-bit PCM**，采样率可在8kHz-44.1kHz之间选择
- 由于ESP32C3的数据存储空间最大为3MByte，因此音频数据的**采样率*截取时间*16bit**必须小于3MByte，请选择合适的采样率和截取时间。



立体声为双声道数据，数据量是单声道的两倍，如果要播放立体声，需要两个功放板和扬声器，同时修改程序中I2S

```
static const i2s_config_t i2s_config = {  
    .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_TX),  
    .sample_rate = 44100,  
    .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,  
    .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,  
    .communication_format = (i2s_comm_format_t)(I2S_COMM_FORMAT_I2S),  
    .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1,  
    .dma_buf_count = 8,  
    .dma_buf_len = 1024,  
    .use_apll=0,  
    .tx_desc_auto_clear= true,  
    .fixed_mclk=-1  
};
```



实验4：素材准备

■ 将.wav转至.h文件

➤ 使用网页文件wav2h进行转换，转换完后会自动下载WavData.h

WAV to Header File Converter

选择文件 王菲-世界赠予我的.wav

Convert to Header (.h)

```
// Auto-generated from WAV file
#ifndef WAV_DATA_H
#define WAV_DATA_H
```

```
unsigned const char WavData[] = {0x52, 0x49, 0x46, 0x46, 0x76, 0x01, 0x0A, 0x00, 0x57, 0x41, 0x56, 0x45, 0x66, 0x6D, 0x74, 0x20,
0x10, 0x00, 0x00, 0x00, 0x01, 0x00, 0x02, 0x00, 0x80, 0xBB, 0x00, 0x00, 0x00, 0xEE, 0x02, 0x00,
0x04, 0x00, 0x10, 0x00, 0x64, 0x61, 0x74, 0x61, 0xFC, 0x00, 0x0A, 0x00, 0x75, 0x11, 0xB5, 0xF4,
0xDF, 0x10, 0xE9, 0xF5, 0xE4, 0x0F, 0xBE, 0xF6, 0x97, 0x0E, 0x43, 0xF7, 0x6F, 0x0D, 0xDF, 0xF7,
0x97, 0x0C, 0xCA, 0xF8, 0x29, 0x0C, 0xEE, 0xF9, 0x4E, 0x0C, 0x37, 0xFB, 0x20, 0x0D, 0xE1, 0xFC,
0x9C, 0x0E, 0xC7, 0xFE, 0xAE, 0x10, 0xD5, 0x00, 0x78, 0x12, 0x5C, 0x02, 0x53, 0x13, 0xEB, 0x02,
0x6D, 0x13, 0x0B, 0x03, 0x95, 0x13, 0xA5, 0x03, 0x1A, 0x14, 0xE4, 0x04, 0xDF, 0x14, 0x88, 0x06,
0x8D, 0x15, 0xFD, 0x07, 0x1E, 0x16, 0x11, 0x09, 0xD7, 0x16, 0x57, 0x0A, 0x62, 0x17, 0xC6, 0x0B,
0x65, 0x17, 0x2F, 0x0D, 0x39, 0x17, 0xD0, 0x0E, 0xBA, 0x16, 0x40, 0x10, 0xA0, 0x15, 0xB2, 0x10,
0x2E, 0x14, 0x84, 0x10, 0x00, 0x13, 0x82, 0x10, 0x28, 0x12, 0xC7, 0x10, 0x30, 0x11, 0xC6, 0x10,
0x1B, 0x10, 0x5E, 0x10, 0x07, 0x0F, 0xB8, 0x0F, 0xD9, 0x0D, 0xAE, 0x0E, 0xAB, 0x0C, 0xAB, 0x0D,
0xAC, 0x0B, 0x35, 0x0D, 0xA9, 0x0A, 0x6D, 0x0D, 0xC0, 0x09, 0x65, 0x0E, 0x83, 0x09, 0x7C, 0x10,
0xFB, 0x09, 0x38, 0x13, 0x2D, 0x0A, 0x89, 0x15, 0x0F, 0x0A, 0x61, 0x17, 0x61, 0x0A, 0x6E, 0x19,
0x54, 0x0B, 0xEB, 0x1B, 0x43, 0x0C, 0x24, 0x1E, 0x92, 0x0C, 0x93, 0x1F, 0x3D, 0x0C, 0x3B, 0x20,
0x80, 0x0B, 0x8C, 0x20, 0xEA, 0x0A, 0x00, 0x21, 0x8D, 0x0A, 0x5D, 0x21, 0x6A, 0x0A, 0x6E, 0x21,
0xDE, 0x0A, 0x6C, 0x21, 0x09, 0x0C, 0xFC, 0x21, 0x6D, 0x0D, 0xCE, 0x22, 0x6C, 0x0E, 0x68, 0x23,
0x84, 0x0F, 0x04, 0x24, 0xFB, 0x10, 0xFC, 0x24, 0x70, 0x12, 0xEE, 0x25, 0xEE, 0x13, 0x95, 0x26,
0x5C, 0x15, 0xFC, 0x26, 0x77, 0x16, 0x00, 0x27, 0xF7, 0x16, 0x94, 0x26, 0x4C, 0x17, 0xC0, 0x25,
0xC2, 0x17, 0xAA, 0x24, 0x5E, 0x18, 0x2C, 0x23, 0x18, 0x19, 0xBE, 0x21, 0x01, 0x1A, 0x6F, 0x20,
0x39, 0x1A, 0xC9, 0x1E, 0x3D, 0x19, 0x81, 0x1C, 0xC3, 0x17, 0x60, 0x1A, 0x88, 0x16, 0xF9, 0x18,
0x90, 0x15, 0x31, 0x18, 0x81, 0x14, 0x82, 0x17, 0x07, 0x13, 0xB8, 0x16, 0x19, 0x11, 0x5D, 0x15,
0xC4, 0x0E, 0x99, 0x13, 0x89, 0x0C, 0xCF, 0x11, 0x84, 0x0A, 0x39, 0x10, 0x78, 0x08, 0x81, 0x0E,
```

➤ 在WAV格式的数据中，前44个子节数据为WAV格式头，之后的数据才是实际的音频波形数据

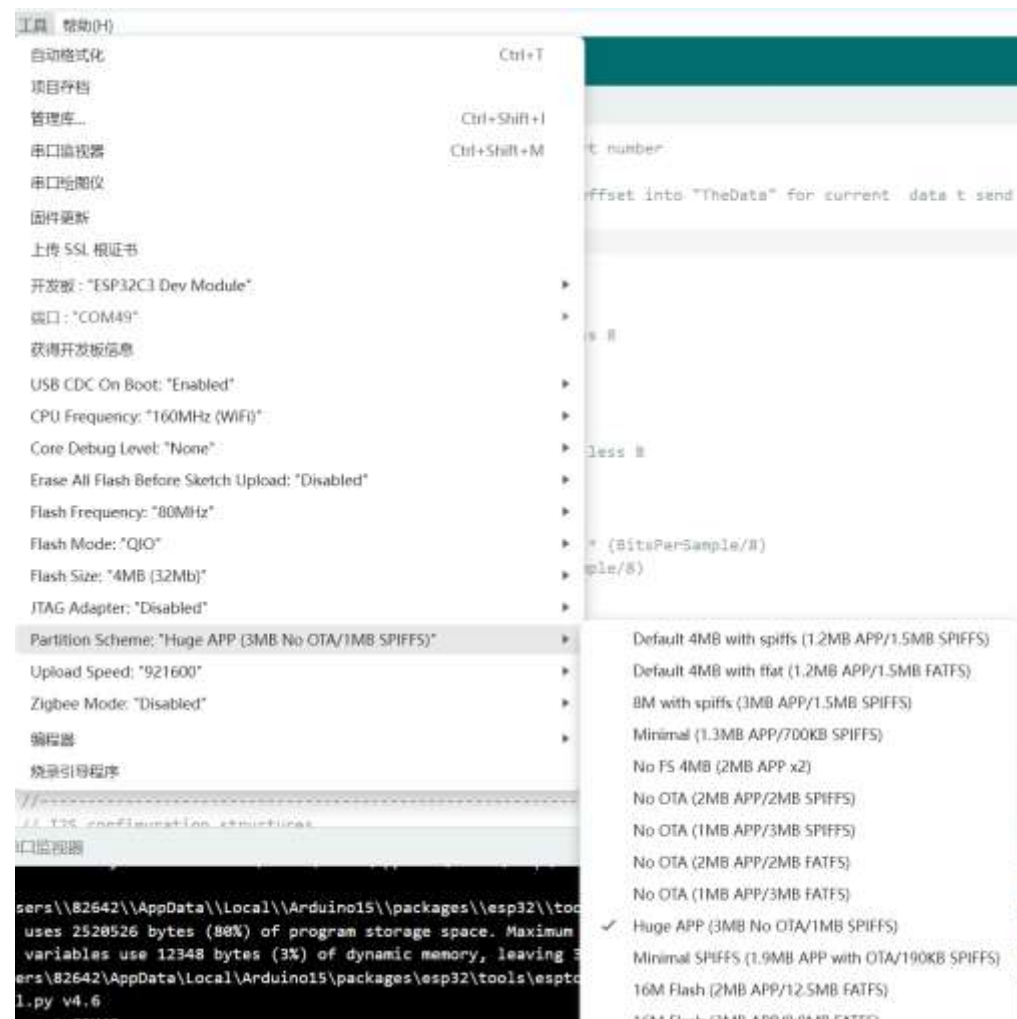


实验4：程序验证

- 用生成的WavData.h替换PlayWav文件夹下的WavData.h，验证上传至esp32c3
- 验证上传前需在Arduino IDE工具中将partition scheme改为Huge App模式，以使能大规模数据的存储
- 可以根据串口监视器的提示，分析生成的.h文件是否正确

```
data ID not present - LIST
```

```
Invlaid data - data section not foundT
```





实验4：实验代码

■ WAV格式的验证

- 从WavData中拷贝44个字节数据到WavHeader结构体变量
- 格式验证无误后配置i2s，这里主要根据数据修改了采样率

```
struct WavHeader_Struct
{
    // RIFF Section
    char RIFFSectionID[4];    // Letters "RIFF"
    uint32_t Size;            // Size of entire file less 8
    char RiffFormat[4];       // Letters "WAVE"

    // Format Section
    char FormatSectionID[4];  // letters "fmt"
    uint32_t FormatSize;      // Size of format section less 8
    uint16_t FormatID;        // 1=uncompressed PCM
    uint16_t NumChannels;     // 1=mono, 2=stereo
    uint32_t SampleRate;      // 44100, 16000, 8000 etc.
    uint32_t ByteRate;        // =SampleRate * Channels * (BitsPerSample/8)
    uint16_t BlockAlign;      // =Channels * (BitsPerSample/8)
    uint16_t BitsPerSample;   // 8, 16, 24 or 32

    // Data Section
    char DataSectionID[4];    // The letters "data"
    uint32_t DataSize;        // Size of the data that follows
}WavHeader;
```

```
memcpy(&WavHeader, &WavData, 44);    // C
DumpWAVHeader(&WavHeader);           // D
if(ValidWavData(&WavHeader))
{
    i2s_driver_install(i2s_num, &i2s_config, 0, NULL);
    i2s_set_pin(i2s_num, &pin_config);
    i2s_set_sample_rates(i2s_num, WavHeader.SampleRate);
    TheData=WavData;
    TheData+=44;
}
```




实验4：实验代码

■ 播放WAV数据

- 使用i2s_write函数每次写4个字节，即单声道2个采样点，立体声（双声道）1个采样点，循环播放

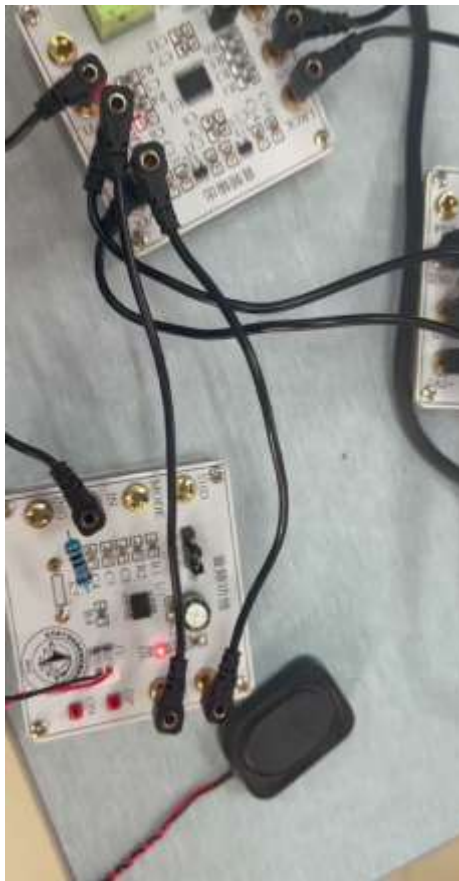
```
void loop()
{
    size_t BytesWritten;                // Returned by the

    // As the WAV data for this example is in form of two 16 bit signe
    i2s_write(i2s_num, TheData+DataIdx, 4, &BytesWritten, portMAX_DELAY);
    DataIdx+=4;                          // increase the data
    if(DataIdx>=WavHeader.DataSize)     // If we gone past e
        DataIdx=0;
}
```




实验4：实验任务

- 修改代码使得可以通过串口监视器输入“1”、“2”、“3”选择播放三种不同的音乐，现场检查





实验5：双声道放音系统

- 在上一实验的基础上，实现双声道同时播放两首音乐的功能，可以通过串口监视器输入“12”、“13”、“23”、“31”...等让左声道播放前一个数字对应的音乐，右声道播放后一个数字对应的音乐。可通过将功放输入连接至音频输出OUTR聆听右声道声音
- 注意要保留要来单音乐播放功能，即输入“1”、“2”、“3”左声道选择播放三种不同的音乐，右声道无输出。可以上一实验一同现场检查